

Homework 1: Molecular dynamics dated September 17, 2026

Note: Computational code must be submitted through Canvas together with appropriate plots and answers to questions (for example as python code or a Jupyter notebook). Answers and short discussion can be submitted as comments inside the code, or as a separate pdf document.

1 Lennard-Jones fluid

Write a molecular dynamics program that implements the velocity-Verlet algorithm, together with a Langevin thermostat, to calculate the equilibrium properties of a Lennard-Jones fluid in 2 dimensions (we here use 2 instead of 3 dimensions to speed up your simulations and allow easier visualization). The Lennard-Jones fluid is described by spherical particles with size σ and a pairwise interaction potential; the interaction between particles i and j is

$$U_{\text{LJ}}(r_{ij}) = 4\epsilon \left[\left(\frac{\sigma}{r_{ij}} \right)^{12} - \left(\frac{\sigma}{r_{ij}} \right)^6 \right] \quad (1)$$

where ϵ is the interaction prefactor and $r_{ij} = |\mathbf{r}_j - \mathbf{r}_i|$ is the distance between particles i and j with $\mathbf{r}_i$ the vector position of particle i . The phase diagram of a 2D Lennard-Jones fluid has been calculated and is given in terms of the reduced temperature $T^* = k_B T / \epsilon$ and number density $\rho^* = \rho \sigma^2$ (see Fig. 1).

Perform the MD simulation of the 2D Lennard-Jones system with periodic boundary conditions (PBC) for a square box of size $10\sigma \times 10\sigma$ containing $N = 70$ particles. Calculate the equilibrium values of the average potential energy per particle:

$$\langle E^{\text{pot}} \rangle = \frac{1}{N} \sum_{i>j} U_{\text{LJ}}(r_{ij}) , \quad (2)$$

and the radial distribution function $g(r)$. Note that a Fourier transform of $g(r)$ gives us the structure factor, which can be directly compared to experimental x-ray or neutron scattering data.

- a) Perform the MD simulation close to the liquid state ($T^* = 0.5$) and plot E^{pot} vs. time. How long (how many time-steps) does it take to equilibrate the system?
- b) Benchmark your simulations and compare with theoretical *Boltzmann distribution*: Measure the velocity distribution of all particles in the equilibrated system and show that velocity along one coordinate, the velocity v_x (i.e. the x component of the velocity) has a Gaussian distribution:

$$p(v_x) = N(0, k_B T / m) , \quad (3)$$

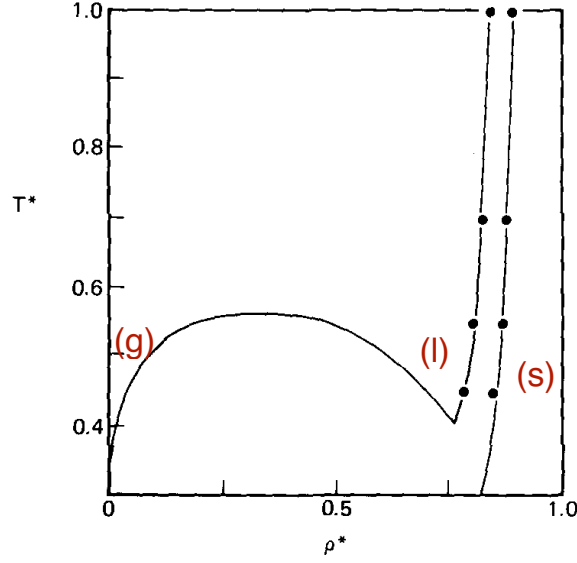


Figure 1: Phase diagram of a 2D Lennard Jones fluid, the area below the line is the coexistence region. The critical point is at $T^* \approx 0.55$, above which the gas and liquid phases become indistinguishable. Taken from J.A. Barker et al, Physica 106a (1981).

where $N(\mu, s^2)$ is a Gaussian (normal) distribution with mean μ and variance s^2 . Also confirm that your simulation reproduces the Boltzmann distribution:

$$p(E_x^{kin}) = \frac{1}{k_B T} e^{-E_x^{kin}/(k_B T)}, \quad (4)$$

where $E_x^{kin} = mv_x^2/2$. Also measure the average kinetic energy per particle and confirm that it follows the equipartition theorem $\langle E^{kin} \rangle = k_B T/2$ per each degree of freedom. Note that this is a 2-dimensional simulation so each particle has two degrees of freedom for velocity: v_x and v_y .

[If your simulation results do not agree with the theory, you may need to run a longer equilibration run to fully equilibrate the system before you start to measure the velocity distribution.]

- c) Calculate the radial distribution function for three cases of your choice of T^* and ρ^* that correspond to three distinct phases on the phase diagram (Fig. 1). Can you identify the phase by the distinct peaks (or lack of them) in the radial distribution function? Plot the equilibrated configuration on a 2D scatter plot (you can just plot one point for each particle center of mass) and calculate E^{pot} for the three cases. Also calculate the diffusivity D of the particles. You can measure diffusivity by measuring the mean square displacement of the particles $\langle R^2 \rangle = 4Dt$ for a 2-dimensional system. Discuss the comparison between $g(r)$, E^{pot} , D and the visualized configurations.

2 Bead–spring polymer

Polymers are typically modeled using bonded interactions together with the Lennard Jones potential for non-bonded interaction; a polymer is represented by a string of beads. Start with the LJ fluid model and add bonded interactions between neighboring beads on a polymer using a harmonic potential

$$U_{\text{harmonic}}(r_{i,i+1}) = \frac{K}{2}(r_{i,i+1} - r_0)^2, \quad (5)$$

where r_0 is the equilibrium bond length. For a polymer of N beads there are a total of $N - 1$ bonds. Note that the Lennard-Jones interaction should be turned off between neighboring particles on the chain: $U_{\text{LJ}}(r_{i,i+1}) = 0$, since the consecutive neighbor interaction is already captured by the bond potential. You may use parameters $r_0 = \sigma$ and $K = 10k_{\text{B}}T/\sigma^2$. Use $N = 50$ beads in a polymer and a box size at least $50\sigma \times 50\sigma \times 50\sigma$ so that the polymer cannot interact with itself through the periodic boundaries. Perform a full three dimensional simulation.

The structure of the polymer is characterized by the radius of gyration, which measures the effective polymer size,

$$R_{\text{g}}^2(t) = \frac{1}{N} \sum_{i=1}^N (\mathbf{r}_i(t) - \bar{\mathbf{r}}(t))^2 \quad (6)$$

where $\bar{\mathbf{r}}(t)$ is the position of the polymer center of mass $\mathbf{r}(t) = \frac{1}{N} \sum_i \mathbf{r}_i(t)$. Note that the value of $R_{\text{g}}(t)$ fluctuates with time, so we will be interested in a time average $R_{\text{g}} = \sqrt{\langle R_{\text{g}}^2(t) \rangle}$. For a flexible polymer, a large R_{g} means that the polymer is in an expanded coil state, for which $R_{\text{g}}^{\text{coil}} \propto r_0 N^\nu$, with $\nu \sim 0.6$ the self-avoiding-walk scaling exponent (note that for a polymer with no repulsive interactions we would find a random-walk exponent $\nu = 0.5$, however, monomer–monomer repulsive interactions change the exponent to 0.6.) Conversely, if the polymer is collapsed to a globule, then R_{g} has a lower value and the volume is proportional to length or $R_{\text{g}}^{\text{globule}} \propto \sigma N^{1/3}$.

- a) Calculate the radius of gyration $g(r)$ of the polymer at three different temperatures $T^* = 2$, $T^* = 5$, $T^* = 0.2$. Can you intuitively explain the result? What happens to the $g(r)$ when you increase the temperature? [For example, protein (un)folding follows a similar pathway. When you cook an egg, the protein albumin gets "denatured" or expanded, which reduces its solubility and causes it to aggregate with other proteins, turning the egg white.]
- b) Calculate the radius of gyration of the polymer for different $N = 10, 20, 50, 100$ at low and high temperature. Explain the result? Does it agree with the theoretical scaling predictions for $R_{\text{g}}^{\text{coil}}$ and $R_{\text{g}}^{\text{globule}}$ given above?

2.1 notes and hints

Note on the required simulation timescale: the equilibration time for a coil polymer is determined by the Rouse time, which calculates the time it takes for the polymer to

diffuse over its own diameter. The diffusivity of the polymer with Langevin dynamics scales as $D(N) = D_0/N$, where D_0 is the diffusion constant for a single monomer. The radius of gyration scales as $R_g \approx r_0 N^\nu$, thus, the Rouse time is obtained by setting $\langle r^2 \rangle = 6Dt = (2R_g)^2$ for which we find $t = 2/3 R_g^2/D \approx N^{2\nu+1} r_0^2/D_0$. Therefore, longer polymers take a significantly longer time to equilibrate; increasing the polymer length by 2 increase the Rouse time by $2^{2.2} \approx 4.6$. In addition, 2x more particles requires 2x more computing power. Thus, a 2x longer polymer requires in total nearly 10x more calculations, and thereby simulating very long polymers (e.g. large proteins) with MD is not efficient. Moreover, monomer-monomer interactions yield kinetic traps that must be overcome to find the correct equilibrated state, which further increase the required MD simulation time. Therefore, for predicting folded structures of large polymers, machine learning methods trained on experimental protein structures, such as alphaFold from Google DeepMind, have been overall been significantly more useful than molecular dynamics.

Hint on implementation: calculating R_g with periodic boundaries can be tricky. Since you are interested in a single isolated polymer, you can just make the box very large (say 3x larger than the polymer) and move the center of mass of the polymer to the origin at every time-step. In MD packages such as LAMMPS, this can be achieved by specifying that the Langevin thermostat does not add any linear momentum to the system, so the center of mass does not move. Alternatively, you can let the polymer map the polymer center of mass